UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

DEFINICIONES Y ARQUITECTURA DE APACHE MAVEN Y EL MODELO DE OBJETOS DE PROYECTO (POM.XML)

Gestión Automatizada de Dependencias, Ciclo de Vida de Construcción y Ecosistema de Software en Java

Presentado por:

Alessandro Villogas Gaspar

Asignatura:

Taller VII: Desarrollo de Aplicaciones I

Docente:

Mg. Ing. Raúl Fernández Bejarano

Huancayo — Perú

2026

Resumen

El presente informe técnico y monográfico aborda los fundamentos conceptuales y metodológicos de Apache Maven como herramienta estándar en la automatización de construcción (build automation) y gestión de dependencias dentro del ecosistema Java. Se examina la filosofía de convención sobre configuración, las fases del ciclo de vida de construcción (default, clean y site) y la anatomía exhaustiva del archivo Project Object Model (pom.xml), destacando las coordenadas GAV (groupId, artifactId, version), el repositorio central y local, y la gestión de plugins. Asimismo, se contextualiza su aplicación práctica dentro del proyecto web empresarial NEXORA implementado bajo Jakarta EE 10 y Java 17, culminando con un análisis comparativo frente a otras herramientas de automatización como Gradle y Apache Ant.

Palabras clave: Apache Maven, POM.xml, dependencias, ciclo de vida, Jakarta EE, build automation, Java 17.

1. Introducción a la Automatización de Construcción

En el desarrollo de software moderno con la plataforma Java, la gestión manual de dependencias mediante la copia física de archivos JAR en carpetas locales representaba históricamente una fuente constante de inconsistencias, errores de versión y conflictos conocidos en la literatura como el 'infierno de dependencias' (JAR Hell). Para mitigar estas limitaciones, surgió Apache Maven, una herramienta de gestión y comprensión de proyectos creada originalmente por Jason van Zyl en 2002 bajo los auspicios de la Apache Software Foundation.

Maven se sustenta en dos principios arquitectónicos medulares: la convención sobre configuración (convention over configuration) y un modelo declarativo y universal para describir la estructura, el ciclo de vida y las dependencias del proyecto mediante un archivo en formato XML denominado Project Object Model (POM).

2. Características Principales de Apache Maven

2.1. Gestión Automática de Dependencias y Repositorios

Maven sustituye la descarga manual de librerías mediante la consulta a repositorios jerárquicos: el repositorio local (~/.m2/repository en el equipo del programador), repositorios

remotos corporativos y el repositorio central público (Maven Central). Al declarar una dependencia, Maven no solo descarga la biblioteca requerida, sino que resuelve transitivamente todas las dependencias hijas que esta necesita para su funcionamiento, garantizando consistencia en las versiones de compilación y ejecución.

2.2. Convención sobre Configuración

Maven estandariza la estructura de directorios en cualquier proyecto Java, permitiendo que cualquier desarrollador se familiarice inmediatamente con el código sin necesidad de scripts personalizados:

```
mi-proyecto/
├── pom.xml                    <- Archivo núcleo de configuración del
proyecto
├── src/
│   ├── main/
│   │   ├── java/              <- Código fuente Java de producción
│   │   ├── resources/         <- Archivos de configuración y propiedades
│   │   └── webapp/            <- Páginas JSP, CSS, JS e imágenes Web
│   └── test/
│       ├── java/              <- Pruebas unitarias automatizadas (JUnit)
│       └── resources/         <- Recursos específicos de entorno de pruebas
└── target/                    <- Salida compilada (clases, archivos .WAR o
.JAR)
```

2.3. Ciclos de Vida de Construcción (Build Lifecycles)

Maven opera sobre tres ciclos de vida predefinidos: 'clean' (limpieza del directorio target), 'default' (compilación, pruebas, empaquetado e instalación) y 'site' (generación de informes y documentación web del proyecto). El ciclo 'default' cuenta con fases secuenciales estrictas:

a) validate: Verifica que el proyecto esté correcto y que toda la información necesaria esté disponible.

b) compile: Compila el código fuente del proyecto ubicado en src/main/java.

c) test: Ejecuta las pruebas unitarias automatizadas (ej. JUnit 5) en src/test/java sin requerir empaquetado previo.

d) package: Empaqueta el código compilado en el formato distribuible declarado (JAR, WAR o EAR).

e) verify: Ejecuta pruebas de integración y verifica criterios de calidad de código.

f) install: Instala el paquete generado en el repositorio local (~/.m2) para su consumo en otros proyectos locales.

g) deploy: Copia el paquete final a un repositorio remoto compartido para despliegue productivo.

3. El Archivo Project Object Model (pom.xml)

El archivo pom.xml representa la unidad fundamental de trabajo en Maven. Contiene información sobre el proyecto y detalles de configuración utilizados por Maven para construirlo. Entre sus elementos clave se encuentran:

• Coordenadas del Proyecto (GAV): groupId (organización o paquete, ej. pe.edu.sistema), artifactId (nombre del módulo, ej. Semana04), version (versión semántica, ej. 1.0-SNAPSHOT) y packaging (tipo de empaquetado, ej. war).

• Propiedades (<properties>): Parámetros globales como la versión de Java (<maven.compiler.source>17</maven.compiler.source>) y la codificación de caracteres (<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>).

• Dependencias (<dependencies>): Declaración de bibliotecas externas (como mssql-jdbc para SQL Server, jakarta.servlet-api para Servlets y junit-jupiter para pruebas unitarias) con sus respectivos ámbitos de alcance (compile, test, provided, runtime).

• Plugins (<build><plugins>): Extensiones encargadas de ejecutar tareas específicas del ciclo de vida, tales como maven-compiler-plugin, maven-war-plugin y maven-surefire-plugin.

4. Implementación Práctica en el Proyecto Web NEXORA

En el marco de la asignatura Taller VII: Desarrollo de Aplicaciones I, el proyecto empresarial NEXORA (Tienda Online de Equipamiento Tecnológico) utiliza Apache Maven con Java 17 y Jakarta EE 10. La configuración del pom.xml orquesta 221 pruebas automatizadas con JUnit 5, el empaquetado en target/Semana04.war para Apache Tomcat 11 y el conector oficial de Microsoft SQL Server:

```
<!-- Fragmento real del pom.xml del proyecto NEXORA -->
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
```

```
    <modelVersion>4.0.0</modelVersion>
    <groupId>pe.edu.sistema</groupId>
    <artifactId>Semana04</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>
    <dependencies>
        <dependency>
            <groupId>jakarta.servlet</groupId>
            <artifactId>jakarta.servlet-api</artifactId>
            <version>6.0.0</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>com.microsoft.sqlserver</groupId>
            <artifactId>mssql-jdbc</artifactId>
            <version>12.8.1.jre11</version>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

5. Comparativa Técnica: Maven frente a Otras Herramientas

A diferencia de Apache Ant (que carece de convenciones y requiere escribir tareas imperativas extensas en XML) y de Gradle (basado en un lenguaje específico de dominio en Groovy/Kotlin con mayor flexibilidad pero curva de aprendizaje más pronunciada), Apache Maven ofrece el balance ideal de robustez, previsibilidad y universalidad en entornos corporativos y educativos, facilitando la integración continua y despliegue continuo (CI/CD).

6. Conclusiones

1. Apache Maven es una pieza arquitectónica indispensable en el desarrollo Java empresarial moderno, estandarizando la estructura de proyectos y resolviendo eficientemente las dependencias transitivas mediante repositorios locales y remotos.

2. El archivo pom.xml centraliza toda la metainformación, garantizando compilaciones idénticas, reproducibles y portables en cualquier entorno operativo (Windows, Linux, macOS) o servidor de integración continua.

3. La aplicación de Maven en el proyecto NEXORA demostró su eficacia al permitir la ejecución ágil de más de 200 pruebas unitarias con surefire y la generación automática del empaquetado WAR para Apache Tomcat 11.

Referencias

Apache Software Foundation. (2024). Maven – Introduction to the Build Lifecycle. https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html

Oracle Corporation. (2023). Java Platform, Standard Edition Documentation (Release 17). Oracle Technical Resources.

Van Zyl, J. (2008). Maven: The Complete Reference. Sonatype Inc.